Authentication and identity: JWT, OAuth2 and the User model in Symfony

Written from: security and lead backend engineering view · **Stack:** PHP 8.4 · Symfony 8.1 · API Platform 5 · Doctrine ORM 3 · PostgreSQL · **Checked:** 12 October 2026 against Packagist, GitHub source, symfony.com, IETF, NIST and OWASP

This tab sets how Colsynq signs people in, issues and checks tokens, models the user, and controls access. It follows the Architecture guide tab: the identity module talks to other modules only through events, and the Domain stays free of framework code. Items marked **(verify)** use syntax not confirmed in a primary source.

1. Decisions in one table

Topic	Decision	Why
Who issues tokens (now)	Colsynq issues its own: **lexik/jwt-authentication-bundle 3.2** for access tokens, **gesdinet/jwt-refresh-token-bundle 3.0** for refresh tokens	Only first-party clients today (web, mobile, admin). Simplest path that is still secure
Who issues tokens (later)	When third parties, banks or open banking need delegated access, move to an **external identity provider** (Keycloak 26 or similar). The API becomes a resource server using Symfony's built-in `oidc` token handler	Building a full OAuth2/OIDC server in-app means writing discovery, JWKS, introspection, PAR, DPoP and mTLS yourself
Partner and machine access	Now: opaque API keys, hashed, via Symfony's `access_token` authenticator. Later: OAuth2 `client_credentials` at the identity provider, with `private_key_jwt` or mTLS	Instant revocation, no shared JWT secret, an easy path to standards

Topic	Decision	Why
Signing	**ES256** with the default (lcobucci) encoder	Small, fast, asymmetric. Rotation by listing old public keys
Access token life	**10 minutes**, 30 s clock skew	Limits damage from a leaked token
Refresh token	Opaque, hashed, **single use with rotation and reuse detection**, 14 days, absolute cap 30 days	Detects theft. Lets users stay signed in
Token subject	`sub` = the user's UUID v7, never the email	Email can change and is personal data
Roles in the token	**None.** The user is reloaded each request; permissions come from the server	Role changes take effect at once
User model	Framework-free `User` aggregate in Domain + a small `SecurityUser` adapter in Infrastructure	Keeps Symfony details out of the Domain
Passwords	Argon2id (`sodium`), migrating from bcrypt; NIST SP 800-63B-4 rules	Bcrypt silently cuts passwords at 72 bytes
Second factor	TOTP + hashed backup codes (scheb/2fa-bundle ^8.6). Passkeys (WebAuthn) for admins and officers	NIST AAL2 needs a phishing-resistant option
Browser apps	Access token in memory, refresh token in an httpOnly, Secure, SameSite=Strict cookie scoped to the refresh path. Back-end-for-frontend for the admin console	Keeps tokens away from scripts (RFC 10017, Aug 2026)
Society (tenant)	User belongs to societies through memberships. Active society chosen per request and checked against membership on the server	One login can serve several societies

2. Packages and versions

Package	Version	Notes
`symfony/security-bundle`	8.1.8	Needs PHP ⩾ 8.4.1
`api-platform/core`	5.0.3	Test base class is now `ApiPlatform\Test\ApiTestCase`
`lexik/jwt-authentication-bundle`	3.2.0	First with Symfony 8. Not tested by its authors against API Platform 5, but the classes it uses still exist. Use `3.x-dev` only if you run FrankenPHP or RoadRunner worker mode (unreleased fix for cached roles)
`gesdinet/jwt-refresh-token-bundle`	3.0.0	PHP 8.4 and Symfony 8 only. Adds reuse detection, session cap, rate limiter. Needs new `family` and `family_valid` columns
`web-token/jwt-library`	4.2.3	Needed for the `oidc` token handler. Install the GMP extension in production
`league/oauth2-server-bundle`	1.2.2 (2.x unreleased)	Only if Colsynq must be its own OAuth2 server. Turn password and implicit grants off explicitly
`knpuniversity/oauth2-client-bundle`	2.20.2	Social login, if done in the API rather than in the identity provider
`symfonycasts/verify-email-bundle`	1.18.0	First with Symfony 8
`symfonycasts/reset-password-bundle`	1.25.0	
`scheb/2fa-bundle`	8.6.1	Require ^8.6: 8.6.0 fixed a timing leak in code checks and a code-reuse cache bug
`web-auth/webauthn-symfony-bundle`	5.3.9	Passkeys
`symfony/rate-limiter`	8.1	For login throttling and refresh limits

Installing any of these needs approval, as with all dependency changes.

3. Choosing who issues tokens

Situation	Use
Our own web, mobile and admin apps sign in with email and password	**lexik + gesdinet** (section 6 and 7)
Tokens come from an external identity provider (Keycloak, Zitadel, Entra, Auth0)	Symfony `access_token` authenticator with the **`oidc`** handler (section 9.1)
A partner system or script calls the API	**Opaque API keys** through `access_token` with a custom handler now (section 9.4); `client_credentials` at the identity provider later
Other companies' apps act for our members (delegated, consent screens)	**External identity provider** as the OAuth2 server. In-app `league/oauth2-server-bundle` only as a last resort (section 9.2)
A browser app on our own domain only	Consider a plain Symfony session or a back-end-for-frontend instead of JWT in the browser

To keep the later move cheap: use the UUID as `sub` from day one, never put roles in tokens, and check permissions with voters. Then switching from lexik to the `oidc` handler is a firewall change, not a code change.

4. The User model

4.1 Two classes, two layers

Class	Layer	Job
`User` aggregate	Domain	Business rules: register, verify email, change password, lock, enable MFA. No Symfony interfaces, no Doctrine attributes
`SecurityUser`	Infrastructure	What Symfony security needs: identifier, roles, password hash. Built from the aggregate. Small and immutable

Class	Layer	Job
`SecurityUserProvider`	Infrastructure	Loads the aggregate through the domain repository and maps it to `SecurityUser`. Also rehashes passwords on login

Domain `User` holds: `UserId` (UUID v7, made in the constructor), `EmailAddress` (lower-case, trimmed, Unicode NFC), `HashedPassword` (opaque string), `UserStatus` (`PendingVerification`, `Active`, `Locked`, `Disabled`), `tokenVersion` (an integer bumped to sign out everywhere), failed-attempt counter and `lockedUntil`, MFA settings, and memberships (society ID + role names).

Domain events it records (translated to integration events, as in the Architecture guide): `UserRegistered`, `EmailVerified`, `PasswordChanged`, `UserLocked`, `UserUnlocked`, `MfaEnabled`, `SessionsRevoked`, `MembershipGranted`, `MembershipRevoked`. Other modules (Notification, Audit, Membership) react to these. They never call the identity module.

4.2 Symfony 8.1 interfaces

- `UserInterface` now has only `getRoles()` and `getUserIdentifier()`. The identifier must be non-empty; an empty one throws.
- `eraseCredentials()` was removed in 8.0. The `security.erase_credentials` option is deprecated in 8.1 and does nothing: delete it. Never keep a plain password on any user object; take it in an input DTO.
- `PasswordAuthenticatedUserInterface` adds `getPassword()`.
- `EquatableInterface::isEqualTo()` decides when a user must be signed out because something changed. Compare id, `tokenVersion`, status and roles, not just the id.
- `UserCheckerInterface::checkPostAuth()` now receives the token. Checkers run on **every request** of a stateless firewall, so keep them fast and free of writes.
- `AttributesBasedUserProviderInterface` lets the provider receive extra data from `UserBadge`, such as the society ID.

4.3 The adapter (sketch, verify)

```
final readonly class SecurityUser implements UserInterface,
PasswordAuthenticatedUserInterface, EquatableInterface
{
    /** @param list<string> $roles platform roles only, e.g.
```

```
ROLE_PLATFORM_ADMIN */
    public function __construct(
        public string $id,              // UUID v7, also the security
identifier
        private ?string $passwordHash,
        private array $roles,
        public int $tokenVersion,
        public bool $active,
    ) {}

    public function getUserIdentifier(): string { return $this->id; }
    public function getRoles(): array { return
array_values(array_unique([...$this->roles, 'ROLE_USER'])); }
    public function getPassword(): ?string { return $this->passwordHash; }

    public function isEqualTo(UserInterface $user): bool
    {
        return $user instanceof self
            && $user->id === $this->id
            && $user->tokenVersion === $this->tokenVersion
            && $user->active === $this->active
            && $user->getRoles() === $this->getRoles();
    }
}

final class SecurityUserProvider implements UserProviderInterface,
PasswordUpgraderInterface
{
    public function __construct(private UserReader $users, private UserWriter
$writer) {}

    public function loadUserByIdentifier(string $identifier): UserInterface
    {
        // Accept a UUID (token requests) or an email (login). Throw
UserNotFoundException if missing.
    }

    public function refreshUser(UserInterface $user): UserInterface { /*
reload by id and map */ }
    public function supportsClass(string $class): bool { return
SecurityUser::class === $class; }

    public function upgradePassword(PasswordAuthenticatedUserInterface $user,
```

```
string $newHashedPassword): void
    {
        // load aggregate, $user->rehashPassword(new
HashedPassword($newHashedPassword)), save
    }
}
```

- Bind the provider in `security.yaml` as a custom provider (`providers: { app: { id: App\...\SecurityUserProvider } }`).
- If a stateful firewall is used (admin console), add `__serialize()` that stores a `crc32c` hash of the password hash instead of the real one. A password change then signs out other sessions.
- Map the aggregate with Doctrine XML in Infrastructure. Doctrine bundle 3 dropped YAML and annotation mapping.
- Column for the hash: `varchar(255)`. Unique index on the normalised email.

4.4 Users, members and societies

- **One identity, many memberships.** A person signs in once and may hold roles in more than one society.
- Platform-wide roles (`ROLE_USER`, `ROLE_PLATFORM_ADMIN`) come from `getRoles()`. Society roles (president, treasurer, unit committee member, member) are **not** Symfony roles like `ROLE_SOCIETY_42_ADMIN`. They are checked by voters against memberships.
- The client names the active society per request (header or path). The server checks a valid membership every time and caches it for the request. A society ID from the client is never trusted alone.
- The member record (person, KYC, employer) belongs to the Membership module. Identity learns of people from `PersonRegisteredV1` and links a login to a person by ID. It does not read Membership tables.
- Roles and permissions per society come from the Access module's events into a local projection that voters read. That keeps the event-only rule.

5. Passwords

```
security:
    password_hashers:

Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
            algorithm: sodium          # Argon2id
            migrate_from: [bcrypt]     # existing bcrypt hashes upgrade on
next login
        recovery_code: 'auto'          # named hasher for backup codes
when@test:
    security:
        password_hashers:

Symfony\Component\Security\Core\User\PasswordAuthenticatedUserInterface:
                algorithm: auto
                cost: 4
                time_cost: 3
                memory_cost: 10
```

Rules from NIST SP 800-63B-4 (final, August 2025):

- Minimum **15 characters** when the password is the only factor; **8** if MFA is always required. Allow at least **64**.
- No composition rules ("one capital, one symbol"), no forced periodic change, no hints, no security questions.
- **Check against breached and context-specific lists** (`NotCompromisedPassword` sends only 5 hex characters of the SHA-1 hash; add words like "colsynq" and the society name).
- Allow paste and password managers. Normalise Unicode to NFC before hashing.
- At most 100 consecutive failures per account.
- Rehash on login happens automatically through `PasswordCredentials` and `PasswordUpgraderInterface`.

6. JWT access tokens (lexik 3.2)

6.1 Install and keys

```
composer require lexik/jwt-authentication-bundle
php bin/console lexik:jwt:generate-keypair
```

- Algorithm: set `encoder.signature_algorithm: ES256` before generating. The default encoder supports only HS, RS and ES families; EdDSA or PS256 need the web-token encoder.
- Keys live in the secrets vault in production, encrypted with a passphrase. Never commit them.
- **Rotation:** publish the new key, keep the old public key in `additional_public_keys` for one token lifetime, then remove it.

6.2 Configuration

```
# config/packages/lexik_jwt_authentication.yaml
lexik_jwt_authentication:
    secret_key: '%env(resolve:JWT_SECRET_KEY)%'
    public_key: '%env(resolve:JWT_PUBLIC_KEY)%'
    pass_phrase: '%env(JWT_PASSPHRASE)%'
    additional_public_keys: []          # old keys during rotation
    token_ttl: 600                      # default 3600
    clock_skew: 30                      # default 0
    allow_no_expiration: false
    encoder:
        signature_algorithm: ES256      # default RS256
    user_id_claim: sub                  # default 'username' would put the
email in every token
    token_extractors:
        authorization_header: { enabled: true, prefix: Bearer, name:
Authorization }
        query_parameter: { enabled: false }   # tokens in URLs leak into logs
    blocklist_token:
        enabled: true                   # OFF by default; also adds the jti
claim
        cache: jwt_blocklist_pool       # must be shared (Redis), not the
filesystem
    api_platform:
        enabled: true                   # documents the login endpoint in
```

```
OpenAPI
        check_path: /api/auth
        username_path: email
        password_path: password
```

Watch out: the bundle's own docs show the blocklist as enabled by default, but the code leaves it off. With it off, tokens have no `jti` and logout cannot withdraw them.

6.3 Claims and checks

The default encoder checks the signature (algorithm fixed by config, so `alg: none` and algorithm-swap attacks fail) and the time claims. It does **not** check `iss`, `aud` or `typ`. Add them:

```
final readonly class JwtClaimsListener
{
    public function __construct(private string $issuer, private string
$audience) {}

    #[AsEventListener(event: Events::JWT_CREATED)]
    public function onCreated(JWTCreatedEvent $event): void
    {
        /** @var SecurityUser $user */
        $user = $event->getUser();
        $data = $event->getData();
        $data['iss'] = $this->issuer;
        $data['aud'] = $this->audience;
        $data['ver'] = $user->tokenVersion;   // sign-out-everywhere check
        unset($data['roles']);                 // permissions come from the
server
        $event->setData($data);

        $header = $event->getHeader();
        $header['typ'] = 'at+jwt';             // RFC 9068
        $event->setHeader($header);
    }

    #[AsEventListener(event: Events::JWT_DECODED)]
    public function onDecoded(JWTDecodedEvent $event): void
    {
        $payload = $event->getPayload();
        if (($payload['iss'] ?? null) !== $this->issuer
            || !in_array($this->audience, (array) ($payload['aud'] ?? []),
```

```
true)) {
            $event->markAsInvalid();
        }
    }
}
```

- Compare the `ver` claim with the reloaded user's `tokenVersion` in a `JWT_AUTHENTICATED` listener or the user checker (verify event name). A mismatch means the user signed out everywhere, changed password, or was disabled.
- Keep the payload small: `sub`, `iss`, `aud`, `iat`, `exp`, `jti`, `ver`. No email, phone, BVN, balances or roles.

6.4 Firewall

```
security:
    providers:
        app_users:
            id: App\Identity\Infrastructure\Security\SecurityUserProvider   #
example path
    firewalls:
        dev:
            pattern: ^/(_profiler|_wdt|assets|build)/
            security: false
        api:
            pattern: ^/api
            stateless: true
            provider: app_users
            user_checker: App\Identity\Infrastructure\Security\UserChecker
            entry_point: jwt
            json_login:
                check_path: /api/auth
                username_path: email
                password_path: password
                success_handler:
lexik_jwt_authentication.handler.authentication_success
                failure_handler:
lexik_jwt_authentication.handler.authentication_failure
            jwt: ~
            refresh_jwt:
                check_path: /api/token/refresh   # required in gesdinet 3
            login_throttling:
                max_attempts: 5
```

```
                interval: '15 minutes'
            logout:
                path: /api/logout
    access_control:
        - { path: ^/api/(auth|token/refresh)$, roles: PUBLIC_ACCESS }
        - { path: ^/api/docs, roles: PUBLIC_ACCESS }
        - { path: ^/api, roles: IS_AUTHENTICATED_FULLY }
```

Routes for `/api/auth`, `/api/token/refresh` and `/api/logout` need to exist (no controller). Behind a load balancer set `trusted_proxies` so throttling sees the real IP.

6.5 Sign-in and refresh flow

```
sequenceDiagram
    autonumber
    participant C as Client app
    participant A as API (/api/auth)
    participant R as API (/api/token/refresh)
    participant X as Any API endpoint
    C->>A: POST email + password
    A->>A: Throttle, load user, check hash, user checker, rehash if needed
    A-->>C: access token (10 min) + refresh token (cookie or body)
    C->>X: Authorization: Bearer access token
    X->>X: Verify signature, iss, aud, exp, ver; reload user; voters
    X-->>C: 200 or 401/403
    C->>R: refresh token (one request at a time)
    R->>R: Hash lookup, single use, reuse check, rotate
    R-->>C: new access token + new refresh token
```

6.6 Where clients keep tokens

Client	Access token	Refresh token
Member web app	In memory only	httpOnly, Secure, SameSite=Strict cookie, path `/api/token/refresh`
Admin and officer console	Server side (back-end-for-frontend); browser holds only a session cookie	Server side
Mobile app	Secure storage (Keychain / Keystore), sent as Bearer	Secure storage, sent in the body

Client	Access token	Refresh token
Never	`localStorage`, `sessionStorage`, URLs	Same

With cookies: strict CORS allow-list, SameSite cookies, and CSRF protection on state-changing requests.

7. Refresh tokens (gesdinet 3.0)

```
gesdinet_jwt_refresh_token:
    refresh_token_class: App\Identity\Infrastructure\Security\RefreshToken
# example
    ttl: 1209600                    # 14 days
    ttl_update: false
    single_use: true                # rotate on every use
    max_session_lifetime: 2592000   # 30-day absolute cap
    max_tokens_per_user: 10         # oldest device dropped
    return_expiration: true
    hash_tokens:
        enabled: true
        accept_stored_in_the_clear: false
    reuse_detection:
        enabled: true               # a spent token coming back revokes the
whole chain
        cache: cache.app            # shared store (Redis)
    block_previous_jwt: true        # needs lexik blocklist
    block_jwts_on_revocation:
        enabled: true
        cache: cache.app
        ttl: 600                    # >= token_ttl
        user_claim: sub             # = lexik user_id_claim
    rate_limiter:
        enabled: true
        limiter: limiter.refresh
        key: ip
    cookie:
        enabled: true
        same_site: strict
        path: /api/token/refresh
        http_only: true
```

```
        secure: true
        remove_token_from_body: true
    api_platform:
        enabled: true
```

- The refresh entity extends the bundle's base class. Add an index on `family`. Generate the migration for `family` and `family_valid`.
- **Reuse detection cannot tell theft from a client racing itself.** The frontend must send only one refresh at a time.
- Cookie settings are global in v3. If mobile needs the token in the body while web uses cookies, handle it in a custom success handler (verify).
- Schedule `gesdinet:jwt:clear` daily with Symfony Scheduler.
- `findAllForUser()` and the session lister support a "your devices" screen; `revokeAllForUser()` signs out everywhere.

8. Revocation and sign-out

Event	What happens
Sign out on one device	Logout listener blocks the current `jti`, revokes that refresh token
Sign out everywhere	`revokeAllForUser()`, bump `tokenVersion`, publish `SessionsRevokedV1`
Password changed or reset	Same as sign out everywhere; email the user
Account locked or disabled	User checker refuses on the next request; bump `tokenVersion`
Refresh token reused	Whole chain revoked; security alert to the user and Audit
Signing key compromised	New key, drop the old one from `additional_public_keys` at once; all users sign in again

Combine: short access tokens, rotating refresh tokens, a `jti` blocklist in Redis, and the per-user `tokenVersion`.

9. OAuth2 and OpenID Connect

9.1 Colsynq as a resource server (tokens from an identity provider)

```
security:
    firewalls:
        api:
            pattern: ^/api
            stateless: true
            access_token:
                token_handler:
                    oidc:
                        claim: sub
                        algorithms: ['ES256', 'RS256']
                        audience: 'colsynq-api'
                        issuers: ['https://auth.example.com/realms/colsynq']
                        discovery:
                            base_uri:
'https://auth.example.com/realms/colsynq/'
                            cache: { id: cache.app }
```

- Needs `web-token/jwt-library` and `symfony/cache`; GMP extension in production.
- Feature history: several algorithms (7.1), discovery and encrypted tokens (7.3), several issuers (7.4), strict key-usage checks (8.1, on by default).
- Map claims to a local user with an `AttributesBasedUserProviderInterface` provider (create the local user on first sight), and map scopes to voters (`members:read`, `payouts:initiate`).
- `security:oidc:generate-token` makes test tokens.
- No built-in token introspection (RFC 7662). For opaque tokens write a custom handler or use `facile-it/php-openid-client`.

9.2 Colsynq as an authorization server

Prefer an external identity provider: Keycloak 26.8, Zitadel v4, Authentik 2026.8, Ory Hydra v26 (headless: login and consent pages stay in Symfony), or a hosted service. They provide OIDC, discovery, JWKS, consent, PAR, client policies and social login.

If it must be in-app: `league/oauth2-server-bundle` 1.2.2 on `league/oauth2-server` 9.4.1.

- Grants: authorization code with PKCE, `client_credentials`, refresh token (rotation), device code.
- **Set `enable_password_grant: false` and `enable_implicit_grant: false` explicitly**; in 1.x they default to on.
- Set `scopes.default`; a client with no scopes otherwise gets every scope.
- Shorten `auth_code_ttl` to one minute.
- Consent screen through the `AUTHORIZATION_REQUEST_RESOLVE` event; scope trimming through `SCOPE_RESOLVE`.
- Gaps you would build yourself: OIDC id_token, discovery, JWKS, introspection, revocation endpoint, PAR, DPoP, mTLS, `private_key_jwt`. Its tokens also lack `iss` and RFC 9068 shape.

9.3 Social login (Google, Apple, Microsoft)

- Best done **inside the identity provider** (brokering), so linking rules live in one place.
- In-app option: `knpuniversity/oauth2-client-bundle` with `league/oauth2-client` providers and the PKCE client.
- **Link accounts on the provider's stable user ID, never just on matching email.** Only trust a verified email from a provider that owns that email domain. The bundle's README example links by email; do not copy it.
- `league/oauth2-client` does not validate id_tokens; validate them yourself.

9.4 Partner and machine access (banks, payroll systems, BaaS callbacks)

Now (API keys):

```
final readonly class ApiKeyHandler implements AccessTokenHandlerInterface
{
    public function __construct(private ApiKeyReader $keys) {}

    public function getUserBadgeFrom(string $accessToken): UserBadge
    {
        $key = $this->keys->findActiveByHash(hash('sha256', $accessToken));
        if (null === $key) {
            throw new BadCredentialsException('Invalid credentials.');
        }

        return new UserBadge($key->clientId(), fn () => $key-
>toSecurityClient());
    }
}
```

- At least 256 random bits, shown once, stored only as a hash, with a prefix for identification (`csk_live_…`).
- One key per partner per environment, scoped (`payroll:remit`, `payments:callback`), with expiry, IP allow-list and audit of every call.
- Inbound webhooks (bank or BaaS confirmations) are verified by **signature (HMAC or JWS) plus timestamp and replay check**, not by bearer tokens alone.

Later (standards): `client_credentials` at the identity provider, short tokens, no refresh tokens, `aud` bound to the API, and client authentication by `private_key_jwt` (RFC 7523) or mTLS (RFC 8705) instead of shared secrets.

Financial-grade: FAPI 2.0 (final, February 2025) asks for PAR, PKCE, sender-constrained tokens (mTLS or DPoP) and strong client authentication. Only realistic through a capable identity provider. There is no Symfony DPoP bundle. The CBN open banking framework (2021, guidelines 2023) prescribes OAuth 2.0; go-live dates are reported inconsistently and the claim that FAPI is mandatory is unconfirmed. Design partner integrations to be FAPI-ready.

10. Account processes

Process	How	Security points
Registration	API Platform `Post` on an input DTO → processor → `RegisterUser` command → `UserRegistered` event	Same response whether or not the email exists ("if this address can be used, we've sent a link"); email the existing owner instead. Throttle by IP and email
Invitation by society	Membership publishes `PersonRegisteredV1`; Identity creates a pending login and sends a one-time link	Link single use, short life, tied to `tokenVersion`
Email verification	verify-email-bundle 1.18, signed link carrying the user ID (works for SPAs)	Link dies when the email changes. Block or limit unverified users in the user checker
Password reset	reset-password-bundle 1.25 (selector + hashed verifier)	`generateFakeResetToken()` when no user, so responses look the same. After reset: bump `tokenVersion`, revoke refresh tokens, notify
MFA	scheb 2fa ^8.6: TOTP, email code, hashed backup codes, trusted device	Over JSON it needs a stateful step: either a session-based firewall for the second step, or a short "MFA pending" token exchanged after the code (verify). Turn on `code_reuse_cache`. Add `PostAuthenticationToken` to `security_tokens`. Set 401/403 yourself; handlers default to 200. Encrypt TOTP secrets at rest
Passkeys	webauthn-symfony-bundle 5.3: `/login/options`, `/login`, opt-in registration	Use the UUID bytes as the user handle, never the email. Required for officers who approve payouts

Process	How	Security points
Magic login links	Core `login_link`	`lifetime: 300`, `max_uses: 1`, `signature_properties: [id, tokenVersion]`, set `framework.trusted_hosts`. Use a confirm page that POSTs, as mail scanners prefetch links
Throttling and lockout	`login_throttling` (Redis-backed) + per-account counter in a `LoginFailureEvent` listener, lock with `LockedException` from the user checker	Throttling is per IP and username; lockout is per account. Cap ⩽ 100 failures. Reset on success. Allow unlock through password reset
Step-up	Re-authentication (password or passkey) within the last 5 minutes	Required for changing email, password, MFA, bank details, and for approving payouts
Impersonation (support)	`switch_user` only on the admin console, gated by a voter	Never across societies, never to a more powerful user, blocked for sensitive actions while `IS_IMPERSONATOR`, every action records both the support user and the impersonated user
Account closure	Command → `UserDeactivated`; data handled as per NDPA retention	Revoke everything; keep audit records

Hide whether an account exists: keep `security.expose_security_errors: none` (the default; `hide_user_not_found` was removed in 8.0). Use the same message and status code for login, registration, reset, verification resend and link requests. When the user is not found in a custom provider, run a dummy hash check so timing does not reveal it.

11. Authorization

- **Platform roles** in `getRoles()` with `role_hierarchy`. Keep the list short.

- **Society permissions** through voters: a `SocietyPermissionVoter` checks that the user has an active membership in the subject's society and that the membership's roles grant the permission (`payout.approve` , `round.open` , `member.admit`).
- Symfony 8 voters: `voteOnAttribute(string $attribute, mixed $subject, TokenInterface $token, ?Vote $vote = null)` ; add reasons with `$vote?->addReason()` . Set voter priority with `#[AsTaggedItem(priority: …)]` (8.1).
- **Separation of duties** (maker and checker) is a voter rule: the approver of a payout cannot be its creator.
- **API Platform 5:** `security` , `securityPostDenormalize` (with `previous_object`), `securityMessage` , and `#[ApiProperty(security: …)]` . **Voters do not filter collections**, so society filtering for lists lives in Doctrine query extensions plus the society SQL filter and row-level security. Return 404, not 403, for objects in another society.
- Workers and CLI: `Security::isGrantedForUser()` checks access without a session.

12. API documentation

- lexik `api_platform.enabled: true` adds the login endpoint to OpenAPI; gesdinet `api_platform.enabled: true` adds refresh.
- Bearer auth button: `api_platform.swagger.api_keys.JWT: { name: Authorization, type: header }` .
- With an identity provider: `api_platform.oauth` with `flow: authorizationCode` , `pkce: true` , and no `clientSecret` in production.

13. Audit and monitoring

- Listen to `LoginSuccessEvent` , `LoginFailureEvent` , `LogoutEvent` , `TokenDeauthenticatedEvent` , `SwitchUserEvent` , the refresh reuse event and MFA events. Publish them as integration events for the Audit module.
- Record: event, user ID, society, IP, user agent, method, impersonator, reason. **Never** passwords, tokens, OTP codes or full emails in logs; mark secrets with `#[SensitiveParameter]` .
- Alert on: refresh token reuse, many failures across accounts from one IP (credential stuffing), MFA disabled, new passkey added, impersonation start.

14. Threats and controls

Threat	Control
`alg: none` or algorithm swap	Algorithm fixed by config; never mix HS with ES/RS
Forged `kid`, `jku`, `x5u`	Keys only from configuration or the identity provider's JWKS; never fetch from token headers
Token theft from the browser	No storage in `localStorage`; short life; back-end-for-frontend for the admin console
Refresh token theft	Rotation, reuse detection, hashing, absolute cap
Token for another API accepted	`aud` and `iss` checks
Credential stuffing	Throttling, breached-password check, MFA, passkeys
Account enumeration	Same responses, fake reset tokens, timing equalised
Cross-society access	Membership check per request, voters, SQL filter, row-level security, 404 responses
Account takeover by social login	Link by provider ID, not email
Stale permissions	No roles in tokens; user reloaded each request; `tokenVersion`
Host header poisoning in emailed links	`framework.trusted_hosts`
Key leak	Vault storage, rotation runbook, short tokens

15. Testing

- Create valid tokens in tests with `JWTTokenManagerInterface::create($user)` (runs the claims listener); create bad ones with `JWTEncoderInterface::encode($claims)`.
- Use `ApiPlatform\Test\ApiTestCase` with `auth_bearer` or `loginUser()`. Fast hashers under `when@test`.

- **Negative tests to keep:** expired token, wrong `aud`, wrong `iss`, tampered signature, `alg: none`, token after logout, token after password change, reused refresh token, refresh rate limit, locked account, unverified account, cross-society object access, collection leak across societies, enumeration responses identical, impersonation blocked for sensitive actions.

16. Checklist

- [ ] `sub` is the UUID; no email or roles in tokens
- [ ] ES256 keys in a vault; rotation runbook tested
- [ ] `token_ttl` 600, `clock_skew` 30, blocklist on with Redis
- [ ] `iss`, `aud`, `typ` and `ver` checked
- [ ] gesdinet: single use, reuse detection, hashing, session cap, rate limit
- [ ] Domain `User` framework-free; `SecurityUser` adapter; no `eraseCredentials()`
- [ ] Argon2id with migration from bcrypt; NIST length and blocklist rules
- [ ] Throttling plus per-account lockout; uniform responses
- [ ] MFA for officers; passkeys for payout approvers; step-up for sensitive changes
- [ ] Society permissions through voters and query extensions
- [ ] Partner keys hashed and scoped; webhooks signed with replay checks
- [ ] Auth events published to Audit; no secrets in logs
- [ ] Negative security tests in CI

17. Sources

- Symfony security and access tokens: https://symfony.com/doc/current/security.html, https://symfony.com/doc/current/security/access_token.html
- LexikJWTAuthenticationBundle: https://github.com/lexik/LexikJWTAuthenticationBundle
- JWTRefreshTokenBundle: https://github.com/markitosgv/JWTRefreshTokenBundle
- API Platform JWT: https://api-platform.com/docs/symfony/jwt/
- league/oauth2-server-bundle: https://github.com/thephpleague/oauth2-server-bundle
- knpuniversity/oauth2-client-bundle: https://github.com/knpuniversity/oauth2-client-bundle

- scheb/2fa: https://github.com/scheb/2fa · WebAuthn bundle: https://github.com/web-auth/webauthn-framework
- RFC 8725 (JWT BCP) and its update draft · RFC 9068 (JWT access tokens) · RFC 9700 (OAuth security BCP, Jan 2025) · RFC 10017 (browser-based apps, Aug 2026) · OAuth 2.1 draft 16
- FAPI 2.0 Security Profile: https://openid.net/specs/fapi-security-profile-2_0-final.html
- NIST SP 800-63B-4: https://pages.nist.gov/800-63-4/sp800-63b.html
- OWASP JWT and Authentication cheat sheets: https://cheatsheetseries.owasp.org/
- Research notes: `doc/research_notes/auth/jwt_symfony.md`, `oauth2_oidc_symfony.md`, `user_entity_symfony.md`